

Daily Problem: 25–Feb–2026

Problem Statement

A *1-counter machine* is a finite automaton equipped with one unbounded integer counter. In one step it may increment the counter, decrement the counter (when it is > 0), and test whether the counter is zero. Show that:

1. A push-down automaton (PDA) with a stack can simulate any 1-counter machine.
2. The converse is false: there are languages accepted by PDAs that cannot be accepted by any 1-counter machine.

Part 1: A PDA can simulate a 1-counter machine

Model

Let the 1-counter machine be

$$M = (Q, \Sigma, \delta, q_0, F),$$

where the transition relation δ depends on:

- the current state $q \in Q$,
- the current input symbol $a \in \Sigma \cup \{\varepsilon\}$,
- the result of the zero-test (i.e. whether the counter value is 0 or > 0),

and each transition updates the counter by $-1, 0, +1$ (with -1 allowed only when the counter is > 0).

We will build a PDA P that recognizes the same language as M .

Encoding the counter in a stack

Use a stack alphabet $\Gamma = \{Z, \#\}$ where Z is the bottom marker and $\#$ represents one unit of counter value.

Invariant:

$$\text{counter value } c \longleftrightarrow \text{stack content is } \#^c Z.$$

So the counter is zero iff the top of stack is Z .

Construction of the PDA

Let

$$P = (Q, \Sigma, \Gamma, \Delta, q_0, Z, F),$$

where the control states and accepting states are the same as M , and the bottom stack symbol is Z .

We simulate each counter instruction:

- **Increment:** push one $\#$ onto the stack.
- **Decrement:** pop one $\#$ from the stack (only possible if top is $\#$, i.e. counter > 0).
- **Zero test:** check whether the top symbol is Z (then counter $= 0$) or $\#$ (then counter > 0).

More precisely, for every transition of M of the form

$$(q, a, \text{isZero}) \rightarrow (q', \text{op}),$$

we add a corresponding PDA transition:

- If $\text{isZero} = \text{true}$, require top-of-stack Z .
- If $\text{isZero} = \text{false}$, require top-of-stack $\#$.
- If $\text{op} = +1$, push $\#$.
- If $\text{op} = 0$, leave the stack unchanged.
- If $\text{op} = -1$, pop $\#$.

Thus the PDA can exactly mimic the counter machine step-by-step, consuming the same input symbols and maintaining the invariant.

Correctness

By induction on the number of simulated steps: after reading any input prefix, P is in the same control state as M , and the number of $\#$ symbols on the stack equals the counter value of M . Therefore, P accepts exactly the strings accepted by M .

Hence every 1-counter language is context-free (indeed, accepted by a PDA).

Part 2: A 1-counter machine cannot simulate every PDA

We show a specific PDA language that no 1-counter machine can accept.

A PDA language that needs more than one “kind” of stack information

Consider the Dyck language with two types of parentheses over

$$\Sigma = \{ (,), [,] \}.$$

Let D_2 be the set of all well-balanced strings with matching types, e.g.

$$([], \quad [()()], \quad ()[[]](), \text{ etc.}$$

It is standard that D_2 is accepted by a PDA: push the opening bracket symbol ($($ or $[$) onto the stack, and when reading a closing bracket, pop and check that the type matches.

Why a 1-counter cannot accept D_2

A 1-counter machine can only store a *single* nonnegative integer (up to finite control), so it can remember at most the *depth* of nesting, not the *sequence of types*. But in D_2 , to verify correctness one must remember the exact *last unmatched opening symbol* at each depth. This requires potentially unbounded information over $\{ (, [\}$, i.e. a true stack alphabet of size ≥ 2 .

We make this precise via Myhill–Nerode style distinguishability within the prefix set.

Define, for each word $u \in \{(\,,\,]\}^*$, the prefix

$$p_u := u$$

(viewed as a string of only opening brackets). For any such u , there is a *unique* closing string $\bar{u}$ (obtained by reversing u and replacing each opening by its matching closing) such that

$$u\bar{u} \in D_2.$$

Example: if $u = ([([, \text{ then } \bar{u} =])])$ (matching types in reverse order).

Now take two different opening strings $u \neq v$ of the same length. Then the two completions $\bar{u}$ and $\bar{v}$ are different, and crucially:

$$u\bar{u} \in D_2 \quad \text{but} \quad v\bar{u} \notin D_2,$$

because when closing, the first mismatch in types forces rejection.

So, to decide membership after reading u (before seeing the closing part), a machine must have enough memory to distinguish all different $u \in \{(\,,\,]\}^n$. There are 2^n such strings of length n , which is exponentially many.

A 1-counter machine, however, after reading n symbols can be in only

$$|Q| \cdot (n + 1)$$

different configurations: the finite control has $|Q|$ states and the counter value is at most n larger than its start value (assuming it changes by at most 1 per step while reading n symbols). This number grows only linearly in n , so for large n it is *much smaller* than 2^n . By the pigeonhole principle, two different opening strings $u \neq v$ of length n must lead the 1-counter machine to the same configuration.

But then appending the same suffix $\bar{u}$ would force the machine to either accept both $u\bar{u}$ and $v\bar{u}$, or reject both, contradicting the fact that exactly one of them is in D_2 . Therefore, no 1-counter machine can recognize D_2 .

Conclusion of Part 2

There exists a PDA language (D_2) that is not recognizable by any 1-counter machine. Hence 1-counter machines are strictly weaker than push-down automata.

Final Conclusion

1. A PDA simulates a 1-counter machine by representing the counter value as the number of stack symbols $\#$ above a bottom marker.
2. The converse fails: PDAs can recognize languages like the Dyck language with two types of parentheses, which require remembering unbounded type information, while a 1-counter machine can store only a single unbounded integer and thus cannot.